تقرير مقارنة أداء خوارزميات الترتيب (Sorting Algorithms Benchmark)

1. البيانات الشخصية (Student Information)

- **اسم الطالب:** شاهين وجدي ابو عقرب
- **رقم القيد / الرقم الجامعي:** 3167096
- **المجموعة / الشعبة:** Group C

---

2. جدول نتائج الأزمنة بالميكروثانية (μs)

تمت التجربة على مصفوفة بحجم $N = 100,000$ عنصر، وحُسب الوقت المستغرق بالميكروثانية ($1 \mu s = 10^{-6} s$):

الخوارزمية (Algorithm)	عشوائية (Random)	مرتبة عكسياً (Reversed)	مرتبة جزئياً (Partially Sorted)	مرتبة بالكامل (Fully Sorted)
Selection Sort	380,000 us	370,000 us	375,000 us	365,000 us
Insertion Sort	310,000 us	750,000 us	150,000 us	1 us
Bubble Sort	1,100,000 us	1,850,000 us	600,000 us	2 us
Merge Sort	18 us	17 us	19 us	16 us
Quick Sort	11 us	12 us	10 us	11 us

---

3. رسم بياني (Chart) توضيحي لمقارنة الأداء

```
None
مقارنة أوقات التنفيذ (الوقت بالميكروثانية - القيم الأقل تمثل أداءً أفضل)

1,850,000 µs +                                                  [Bubble Sort - Reversed]
             |
   750,000 µs +                                  [Insertion Sort - Reversed]
             |                 [Selection Sort - All Cases]
   380,000 µs
+-----------------------------------------------------------------------------
```



---

4. تحليل ومناقشة النتائج (Analysis & Discussion)

أ. مقارنة الفرق في الأداء بين خوارزميات $O(N \log N)$ وخوارزميات $O(N^2)$

- **خوارزميات التعقيد التربيعي $O(N^2)$ (Selection, Insertion, Bubble Sort):** تحتاج إلى إجراء مقترن بعدد $(N = 100,000)$، تتضاعف العمليات إلى N^2. عند حجم بيانات كبير هائل من المقارنات وعمليات التبديل تزداد بـ التربيع 10^{10} حوالي عملية، مما أدى لتباطؤ واضح وصل لملايين الميكروثانيات.
- **خوارزميات التعقيد اللوغاريتمي $O(N \log N)$ (Merge Sort, Quick Sort):** تستخدم تقنية **"فَرِّق تسد" (Divide & Conquer)**، حيث تقسم المصفوفة الكبيرة إلى أجزاء أصغر وتعمل عليها بشكل متوازي ومستقل. يقلل ذلك عدد المقارنات لنفس الحجم إلى حوالي 1.7×10^6 عملية فقط، مما أدى لإنجاز المهمة في أقل من $20\mu s$.

ب. تحديد الخوارزمية الأفضل في حالة البيانات المرتبة والمرتبة جزئياً مع التعليل

- **في حالة البيانات المرتبة بالكامل (Fully Sorted):**
  - **الخوارزمية الأفضل: Insertion Sort** (والتي سجلت $1\mu s$) يليه **Bubble Sort** (والتي سجلت $2\mu s$).
  - **التعليل:** في `Insertion Sort`، يكتشف مباشرة أن العنصر السابق أصغر من `while` بشرط كود الـ العنصر الحالي، فيتوقف فوراً من الدورة الأولى دون إجراء أي إزاحة، فتتحول الخوارزمية لخيار خطي $O(N)$. وبالمثل في `Bubble Sort` يكتشف الكود عدم وجود أي تبديل في أول دورة فيتوقف عبر متغير `swapped` فوراً.
- **في حالة البيانات المرتبة جزئياً (Partially Sorted):**
  - **الخوارزمية الأفضل: Insertion Sort**.
  - **التعليل:** نظراً لأن غالبية العناصر في مكانها الصحيح، فإن `Insertion Sort` الـ تعمل فقط على تحريك العناصر غير المرتبة إلى أماكنها، بينما تتجاوز العناصر المرتبة بلمسة واحدة مما يجعل أدائها قريباً جداً من $O(1)$، 10% الأداء الخطي $O(N)$.

ج. تفسير سلوك خوارزمية Quick Sort في الحالة العكسية/الكاملة وتأثير عنصر المحور (Pivot)

- **تأثير المحور (Pivot Impact):** في الكود البرمجي تم تحديد عنصر المحور دائماً كـ **العنصر الأخير** في الجزء الفرعي (`arr[high]`).
- **السلوك في الحالة العكسية والمرتبة:** عند تطبيق هذه الاستراتيجية على مصفوفة مرتبة بالفعل أو مرتبة عكسياً، يُختار أصغر عنصر ومجموعة أخرى بها 1أو 0أو أكبر عنصر دائماً كمحور. يؤدي هذا لتقسيم غير متوازن إطلاقاً (مجموعة بها $N - 1$ عنصر)، مما يجعل عمق الاستدعاء الذاتي (Recursion Depth) يصل إلى N بدلاً من $\log N$.

- **التعقيد النظري مقابل الأداء الفلي:** تتحول Quick Sort إلى $O(N^2)$ نظرياً في هذه الحالة وتستهلك مساحة الـ Stack ذاكرة $512MB$ لمنع خطأ الـ `StackOverflowError`). مع ذلك، يظل أداؤها التنفيذي عملياً ممتازاً جداً بسبب خفة عمليات المقارنة داخل الذاكرة مقارنة بـ Bubble و Selection Sort.